

R-Code Beispiele aus WDDA Vorlesung 4 (Kapitel 4)

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-04-29

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Notation: Population vs. Stichprobe	2
4	Simulation der Stichprobenvariation	4
5	Standardfehler der Stichprobenmittelwerte	6
6	Einfluss der Stichprobengrösse	6
7	Bereich plausibler Werte	11
8	Konfidenzintervalle – Simulation für Proportionen	12
9	Darstellung von Konfidenzintervallen aus der Simulation	14
10	Bootstrap-Stichproben und Standardfehler	15
11	Bootstrap: Beispiel – Nussmischung	18
12	Konfidenzintervalle mit der Bootstrap-Methode (SE-Methode)	20
13	Alternative Konfidenzintervalle – Bootstrap-Perzentil-Methode	21
14	Konfidenzintervalle mit Perzentilen für unterschiedliche Konfidenzniveaus	22
15	Konfidenzintervalle für Differenzen	24
16	Konfidenzintervalle für Korrelationen	26
17	Zusammenfassung	29

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der Folien aus der WDDA FS2026 Vorlesung 4 (Kapitel 4) zusammen. Der Fokus liegt auf der schliessenden Statistik, insbesondere auf Stichprobenvariation, Standardfehlern, Bootstrap-Methoden und Konfidenzintervallen.

Die schliessende Statistik (inferentielle Statistik) ist ein Bereich der Statistik, der Schlussfolgerungen über eine Grundgesamtheit auf Basis von Stichproben zieht. Im Gegensatz zur deskriptiven Statistik, die “nur” die vorliegenden Daten beschreibt, versucht die schliessende Statistik, Aussagen über die Gesamtpopulation zu treffen.

Zentrale Konzepte in diesem Bereich sind:

1. **Stichprobenvariation:** Wie stark variieren Statistiken (z.B. Mittelwerte) zwischen verschiedenen Stichproben?
2. **Standardfehler:** Ein Mass für die Präzision einer Stichprobenstatistik
3. **Bootstrap-Methoden:** Resampling-Techniken zur Schätzung von Standardfehlern und Konfidenzintervallen
4. **Konfidenzintervalle:** Bereiche, die mit einer bestimmten Wahrscheinlichkeit den wahren Populationsparameter enthalten

2 Vorbereitung

```
# Pakete laden
library(mosaic)      # z.B. für do(), resample(), dotPlot()
library(plotrix)     # für plotCI()
library(readxl)      # zum Einlesen von Excel-Dateien

# Daten importieren
Footballer <- read_excel("data/WDDA_04.xlsx", sheet = "Footballer")
BFH <- read_excel("data/WDDA_04.xlsx", sheet = "BFH")
ExerciseHours <- read_excel("data/WDDA_04.xlsx", sheet = "ExerciseHours")
Mustangs <- read_excel("data/WDDA_04.xlsx", sheet = "Mustangs")

# Überprüfen der Datenstruktur
str(Footballer)

## tibble [581 x 2] (S3: tbl_df/tbl/data.frame)
##   $ Player      : chr [1:581] "Gareth Bale" "David De Gea" "Kevin De Bruyne" "Raheem Sterling" ...
##   $ WeeklySalary: num [1:581] 600000 375000 320833 300000 290000 ...
```

R-Erklärung:

- Die Funktion `library()` lädt ein installiertes Paket in die aktuelle R-Sitzung.
- Mit `read_excel()` aus dem Paket `readxl` können Excel-Dateien eingelesen werden. Der Parameter `sheet` gibt an, welches Tabellenblatt verwendet werden soll.
- Die Funktion `str()` zeigt die Struktur eines Objekts, einschliesslich Datentypen und Dimensionen.

Hinweis zu Datenpfaden: In R-Markdown-Dateien werden relative Pfade vom Speicherort der Datei aus interpretiert, daher der Pfad `"../data/WDDA_04.xlsx"`. In regulären R-Skripten würde der Pfad relativ zum Arbeitsverzeichnis angegeben werden.

3 Notation: Population vs. Stichprobe

In der Statistik unterscheiden wir zwischen Parametern der Population und Schätzungen aus Stichproben. Diese werden mit unterschiedlichen Symbolen notiert:

- **Populationsparameter:** z.B. Mittelwert μ , Standardabweichung σ , Varianz σ^2 , Proportion p
- **Stichprobenschätzungen:** z.B. Mittelwert $\bar{x}$, Standardabweichung s , Varianz s^2 , Proportion $\hat{p}$

```

# Extrahieren der wöchentlichen Gehälter
WeeklySalary <- Footballer$WeeklySalary

# Berechnung des Populationsmittelwerts (hier: alle Fussballspieler in der Datei)
mu <- mean(WeeklySalary)

# Ziehen einer Stichprobe und Berechnung des Stichprobenmittelwerts
set.seed(123) # Für Reproduzierbarkeit
stichp1 <- sample(WeeklySalary, size = 30)
xbar <- mean(stichp1)

# Ausgabe der Ergebnisse
cat("Populationsmittelwert ( $\mu$ ):", round(mu, 2), "\n")

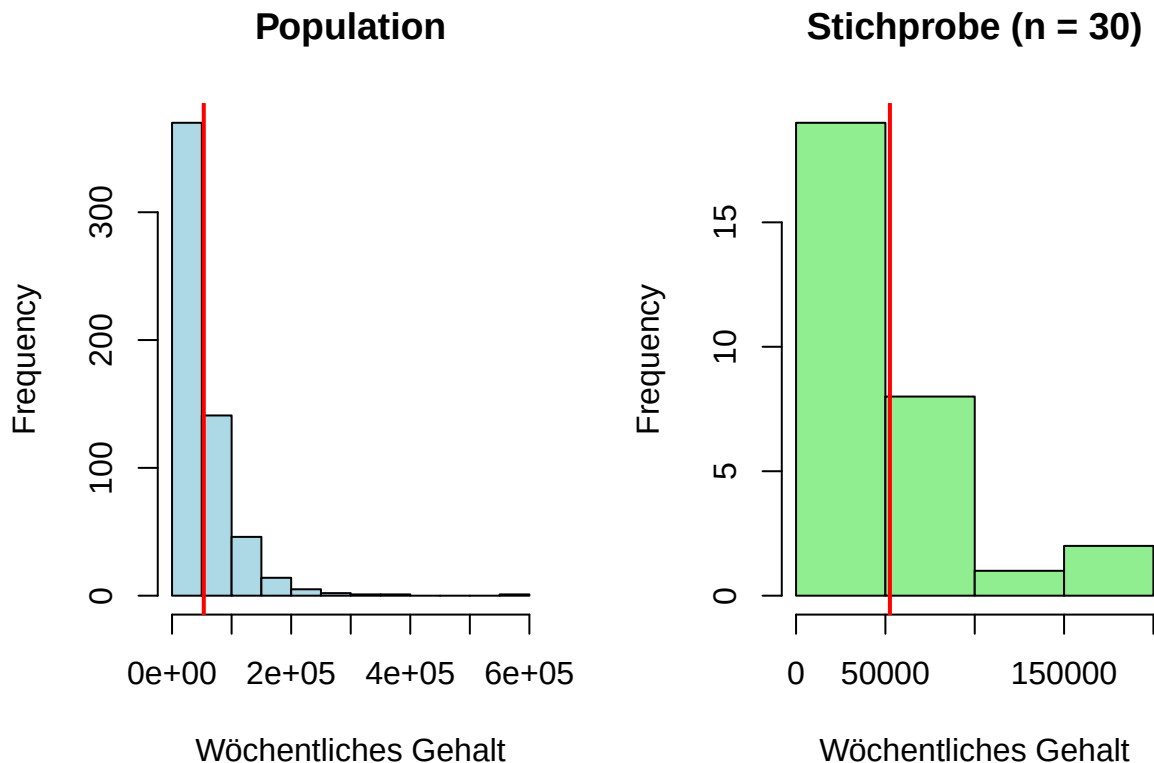
## Populationsmittelwert ( $\mu$ ): 53280.81

cat("Stichprobenmittelwert ( $\bar{x}$ ) (n = 30):", round(xbar, 2), "\n")

## Stichprobenmittelwert ( $\bar{x}$ ) (n = 30): 52519.8

# Visualisierung
par(mfrow = c(1, 2))
hist(WeeklySalary, main = "Population", xlab = "Wöchentliches Gehalt", col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
hist(stichp1, main = "Stichprobe (n = 30)", xlab = "Wöchentliches Gehalt", col =
  ↪ "lightgreen")
abline(v = xbar, col = "red", lwd = 2)

```



```
par(mfrow = c(1, 1))
```

Statistische Erklärung:

- Der **Populationsmittelwert** ist der wahre Mittelwert aller Elemente in der Grundgesamtheit.
- Der **Stichprobenmittelwert** $\bar{x}$ ist eine Schätzung von μ , basierend auf einer zufälligen Teilmenge der Population.
- In der Praxis kennen wir μ oft nicht und müssen es anhand von $\bar{x}$ schätzen.
- Verschiedene Stichproben führen zu unterschiedlichen Werten von $\bar{x}$, was als **Stichprobenvariabilität** bezeichnet wird.

R-Erklärung:

- Die Funktion `sample()` zieht eine zufällige Stichprobe aus einem Vektor. Der Parameter `size` gibt die Stichprobengröße an.
- Die Funktion `set.seed()` setzt den Startwert für den Zufallszahlengenerator, was die Reproduzierbarkeit der Ergebnisse gewährleistet.
- Die Funktion `abline()` mit dem Parameter `v` fügt eine vertikale Linie zum Plot hinzu.

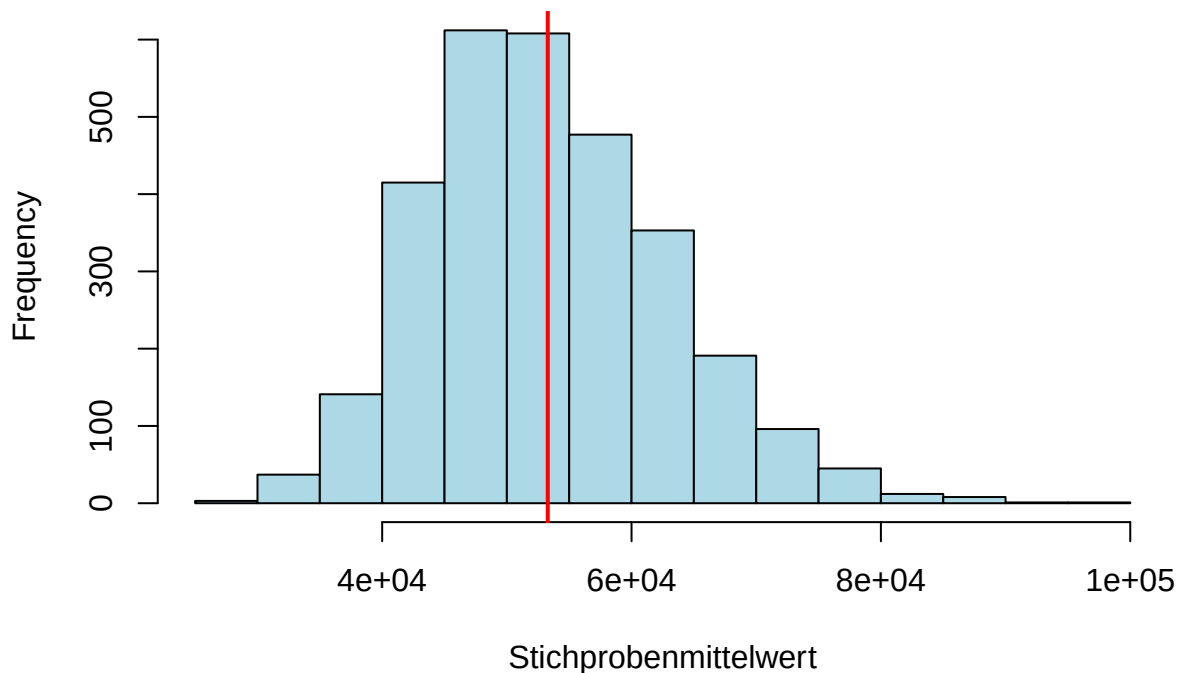
4 Simulation der Stichprobenvariation

Um die Variation zwischen verschiedenen Stichproben zu verstehen, können wir viele Stichproben ziehen und die Verteilung der Stichprobenmittelwerte betrachten.

```
# Ziehe 3000 Stichproben (n = 30) und berechne jeweils den Mittelwert
set.seed(456)
m.stichps <- do(3000) * mean(sample(WeeklySalary, size = 30))

# Visualisierung der Verteilung der Stichprobenmittelwerte
hist(m.stichps$mean,
     main = "Verteilung der Stichprobenmittelwerte (n = 30)",
     xlab = "Stichprobenmittelwert",
     col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
```

Verteilung der Stichprobenmittelwerte (n = 30)



```
# Alternative Visualisierung mit dotPlot
dotPlot(m.stichps$mean,
  nint = 200,
  cex = 0.5,
  pch = 16,
  ylim = c(0, 100),
  main = "Verteilung der Stichprobenmittelwerte (n = 30)",
  xlab = "Stichprobenmittelwert")
abline(v = mu, col = "red", lwd = 2)
```

Statistische Erklärung:

- Die Verteilung der Stichprobenmittelwerte wird als **Stichprobenverteilung** bezeichnet.
- Der **zentrale Grenzwertsatz** besagt, dass die Stichprobenverteilung des Mittelwerts bei ausreichend grosser Stichprobengrösse annähernd normalverteilt ist, unabhängig von der Form der Populationsverteilung.
- Der Mittelwert der Stichprobenverteilung entspricht dem Populationsmittelwert (Erwartungstreue).
- Die Standardabweichung der Stichprobenverteilung wird als **Standardfehler** bezeichnet.

R-Erklärung:

- Die Funktion `do()` aus dem Paket `mosaic` wiederholt einen Ausdruck mehrmals.
- Der Operator `*` in `do(3000) * mean(...)` gibt an, dass der Ausdruck 3000 Mal wiederholt werden soll.
- Die Funktion `dotPlot()` erstellt ein Punktdiagramm, wobei jeder Punkt einem Stichprobenmittelwert entspricht.

5 Standardfehler der Stichprobenmittelwerte

Der Standardfehler ist ein Mass für die Präzision einer Stichprobenstatistik. Er gibt an, wie stark die Stichprobenstatistik (z.B. der Mittelwert) zwischen verschiedenen Stichproben variiert.

```
# Berechnung des Standardfehlers aus der Simulation
se_simuliert <- sd(m.stichps$mean)
cat("Simulierter Standardfehler (se):", round(se_simuliert, 2), "\n")

## Simulierter Standardfehler (se): 9662.53

# Theoretischer Standardfehler nach der Formel  $se = \sigma / \sqrt{n}$ 
sigma <- sd(WeeklySalary) # Populationsstandardabweichung
n <- 30 # Stichprobengröße
se_theoretisch <- sigma / sqrt(n)
cat("Theoretischer Standardfehler (se):", round(se_theoretisch, 2), "\n")

## Theoretischer Standardfehler (se): 9931.83

# Vergleich der Variation einzelner Werte mit der Variation der Mittelwerte
cat("Standardabweichung der Population ():", round(sigma, 2), "\n")

## Standardabweichung der Population (): 54398.89

cat("Standardfehler des Mittelwerts (se):", round(se_simuliert, 2), "\n")

## Standardfehler des Mittelwerts (se): 9662.53

cat("Verhältnis  $\sigma/se$ :", round(sigma/se_simuliert, 2),
    "(entspricht ungefähr  $\sqrt{n} =$ ", round(sqrt(n), 2), ") \n")

## Verhältnis  $\sigma/se$ : 5.63 (entspricht ungefähr  $\sqrt{n} = 5.48$  )
```

Statistische Erklärung:

- Der **Standardfehler (SE)** ist die Standardabweichung der Stichprobenverteilung einer Statistik.
- Für den Mittelwert gilt: $SE = \sigma / \sqrt{n}$, wobei σ die Populationsstandardabweichung und n die Stichprobengröße ist.
- Der Standardfehler nimmt mit zunehmender Stichprobengröße ab (proportional zu $1/\sqrt{n}$).
- Der Standardfehler ist ein wichtiges Mass für die Präzision einer Schätzung: Je kleiner der Standardfehler, desto präziser die Schätzung.
- In der Praxis kennen wir oft nicht σ und müssen den Standardfehler mit $s/\sqrt{n}$ schätzen, wobei s die Stichprobenstandardabweichung ist.

R-Erklärung:

- Die Funktion `sd()` berechnet die Standardabweichung eines Vektors.
- Die Funktion `sqrt()` berechnet die Quadratwurzel einer Zahl.

6 Einfluss der Stichprobengröße

Die Stichprobengröße hat einen entscheidenden Einfluss auf die Präzision von Schätzungen. Mit zunehmender Stichprobengröße wird die Variation der Stichprobenstatistiken kleiner.

```
# Ziehe Stichproben unterschiedlicher Größe und berechne jeweils den Mittelwert
set.seed(789)
m_30 <- do(3000) * mean(sample(WeeklySalary, size = 30))
m_100 <- do(3000) * mean(sample(WeeklySalary, size = 100))
m_500 <- do(3000) * mean(sample(WeeklySalary, size = 500))
```

```
# Berechnung der Standardfehler
se_30 <- sd(m_30$mean)
se_100 <- sd(m_100$mean)
se_500 <- sd(m_500$mean)
```

```
# Ausgabe der Standardfehler
cat("Standardfehler (n = 30):", round(se_30, 2), "\n")
```

```
## Standardfehler (n = 30): 9602.82
```

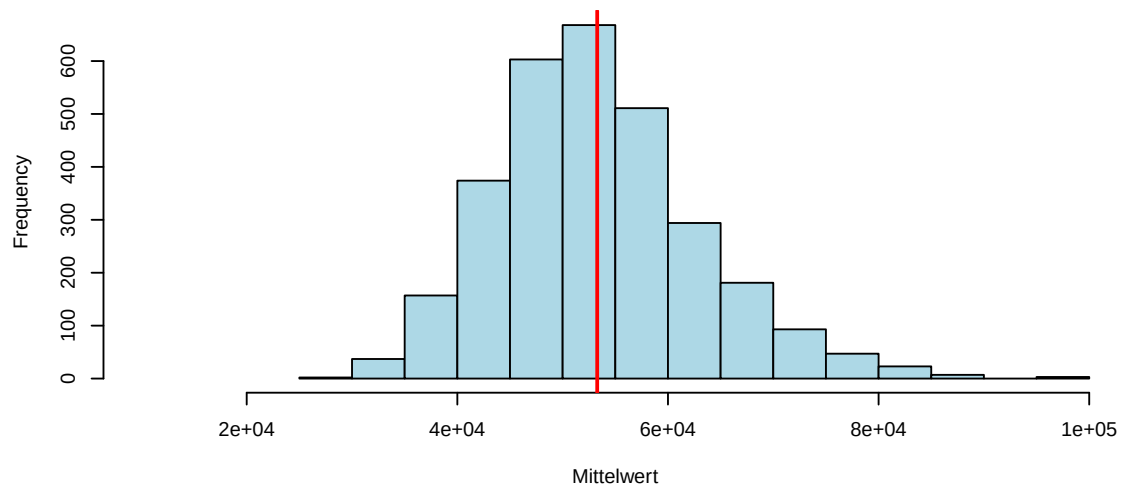
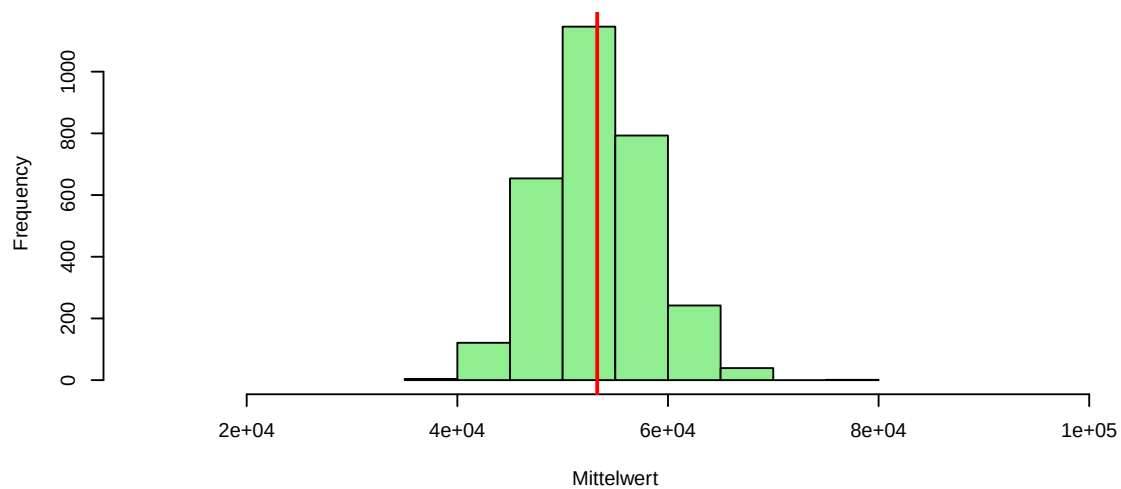
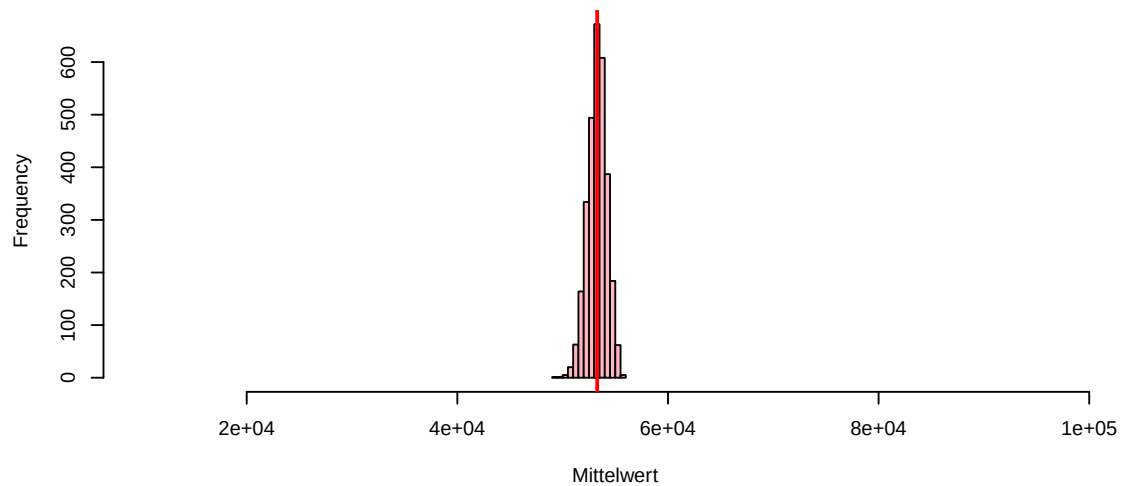
```
cat("Standardfehler (n = 100):", round(se_100, 2), "\n")
```

```
## Standardfehler (n = 100): 4940.48
```

```
cat("Standardfehler (n = 500):", round(se_500, 2), "\n")
```

```
## Standardfehler (n = 500): 903.14
```

```
# Visualisierung der Verteilungen
par(mfrow = c(3, 1))
hist(m_30$mean, xlim = c(10000, 100000),
     main = "Stichprobenmittelwerte (n = 30)",
     xlab = "Mittelwert",
     col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
hist(m_100$mean, xlim = c(10000, 100000),
     main = "Stichprobenmittelwerte (n = 100)",
     xlab = "Mittelwert",
     col = "lightgreen")
abline(v = mu, col = "red", lwd = 2)
hist(m_500$mean, xlim = c(10000, 100000),
     main = "Stichprobenmittelwerte (n = 500)",
     xlab = "Mittelwert",
     col = "lightpink")
abline(v = mu, col = "red", lwd = 2)
```

Stichprobenmittelwerte (n = 30)**Stichprobenmittelwerte (n = 100)****Stichprobenmittelwerte (n = 500)**

```
par(mfrow = c(1, 1))
```

```
# Vergleich der theoretischen und simulierten Standardfehler
```

```
se_theo_30 <- sigma / sqrt(30)
```

```
se_theo_100 <- sigma / sqrt(100)
```

```
se_theo_500 <- sigma / sqrt(500)
```

```
cat("\nVergleich theoretischer und simulierter Standardfehler:\n")
```

```
##
```

```
## Vergleich theoretischer und simulierter Standardfehler:
```

```
cat("n = 30: Theoretisch:", round(se_theo_30, 2),  
    "Simuliert:", round(se_30, 2), "\n")
```

```
## n = 30: Theoretisch: 9931.83 Simuliert: 9602.82
```

```
cat("n = 100: Theoretisch:", round(se_theo_100, 2),  
    "Simuliert:", round(se_100, 2), "\n")
```

```
## n = 100: Theoretisch: 5439.89 Simuliert: 4940.48
```

```
cat("n = 500: Theoretisch:", round(se_theo_500, 2),  
    "Simuliert:", round(se_500, 2), "\n")
```

```
## n = 500: Theoretisch: 2432.79 Simuliert: 903.14
```

```
# Visualisierung des Zusammenhangs zwischen n und SE
```

```
n_values <- c(30, 100, 500)
```

```
se_values <- c(se_30, se_100, se_500)
```

```
plot(n_values, se_values,  
     type = "b",  
     log = "x", # Logarithmische x-Achse  
     main = "Standardfehler in Abhängigkeit von der Stichprobengröße",  
     xlab = "Stichprobengröße (n)",  
     ylab = "Standardfehler (SE)",  
     col = "blue",  
     pch = 16)  
# Theoretische Kurve: SE = /√n  
curve(sigma / sqrt(x), add = TRUE, col = "red", lty = 2, from = 30, to = 500)  
legend("topright",  
     legend = c("Simulierte Werte", "Theoretische Kurve ( /√n)"),  
     col = c("blue", "red"),  
     lty = c(1, 2),  
     pch = c(16, NA))
```

```
## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,  
## : conversion failure on 'Theoretische Kurve ( /√n)' in 'mbcsToSbcs': dot  
## substituted for <cf>
```

```
## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,  
## : conversion failure on 'Theoretische Kurve ( /√n)' in 'mbcsToSbcs': dot
```

```
## substituted for <83>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretische Kurve (/√n)' in 'mbcsToSbcs': dot
## substituted for <e2>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretische Kurve (/√n)' in 'mbcsToSbcs': dot
## substituted for <88>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretische Kurve (/√n)' in 'mbcsToSbcs': dot
## substituted for <9a>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretische Kurve
## (/√n)' in 'mbcsToSbcs': dot substituted for <cf>

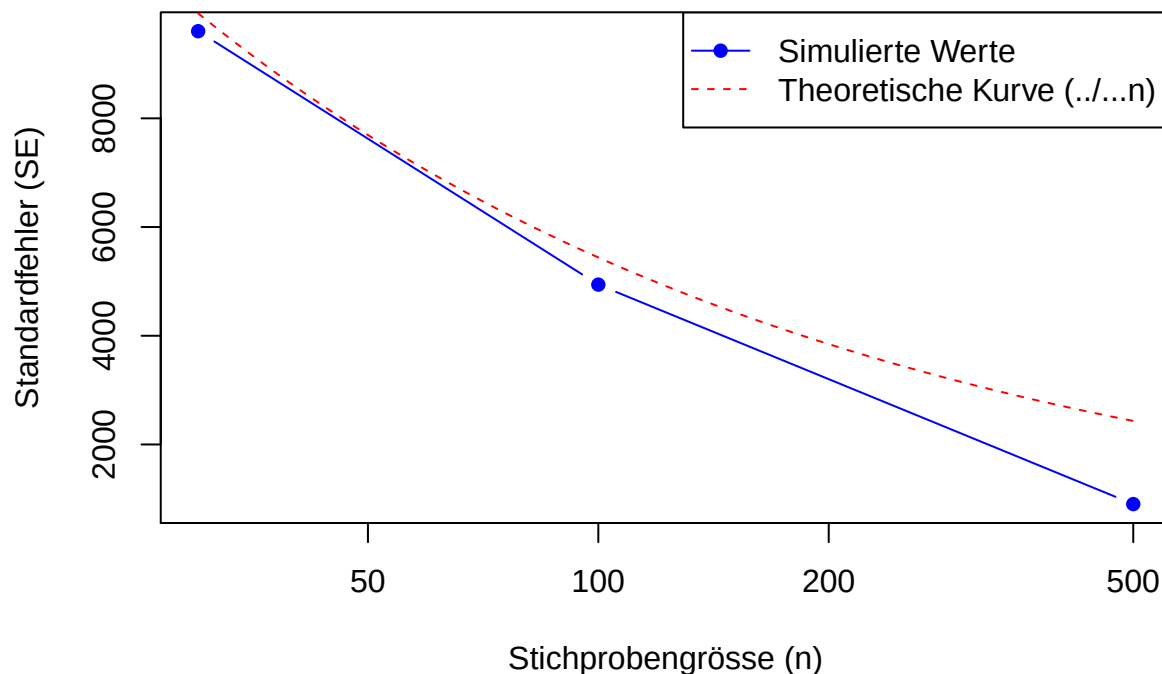
## Warning in text.default(x, y, ...): conversion failure on 'Theoretische Kurve
## (/√n)' in 'mbcsToSbcs': dot substituted for <83>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretische Kurve
## (/√n)' in 'mbcsToSbcs': dot substituted for <e2>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretische Kurve
## (/√n)' in 'mbcsToSbcs': dot substituted for <88>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretische Kurve
## (/√n)' in 'mbcsToSbcs': dot substituted for <9a>
```

Standardfehler in Abhängigkeit von der Stichprobengröße



Statistische Erklärung:

- Mit zunehmender Stichprobengröße n :
 - Wird die Verteilung der Stichprobenmittelwerte schmäler (geringere Streuung)

- Nimmt der Standardfehler ab (proportional zu $1/\sqrt{n}$)
- Werden die Schätzungen präziser
- Die Beziehung $SE = 1/\sqrt{n}$ zeigt, dass der Standardfehler mit der Quadratwurzel der Stichprobengröße abnimmt.
- Dies bedeutet, dass für eine Halbierung des Standardfehlers die Stichprobengröße vervierfacht werden muss.
- Dieser abnehmende Grenznutzen erklärt, warum in der Praxis oft Stichprobengrößen zwischen 30 und 1000 verwendet werden.

R-Erklärung:

- Der Parameter `xlim` in `hist()` legt die Grenzen der x-Achse fest.
- Die Funktion `curve()` zeichnet eine Funktion über einen bestimmten Bereich.
- Der Parameter `log = "x"` in `plot()` erstellt eine logarithmische x-Achse.
- Die Funktion `legend()` fügt eine Legende zum Plot hinzu.

7 Bereich plausibler Werte

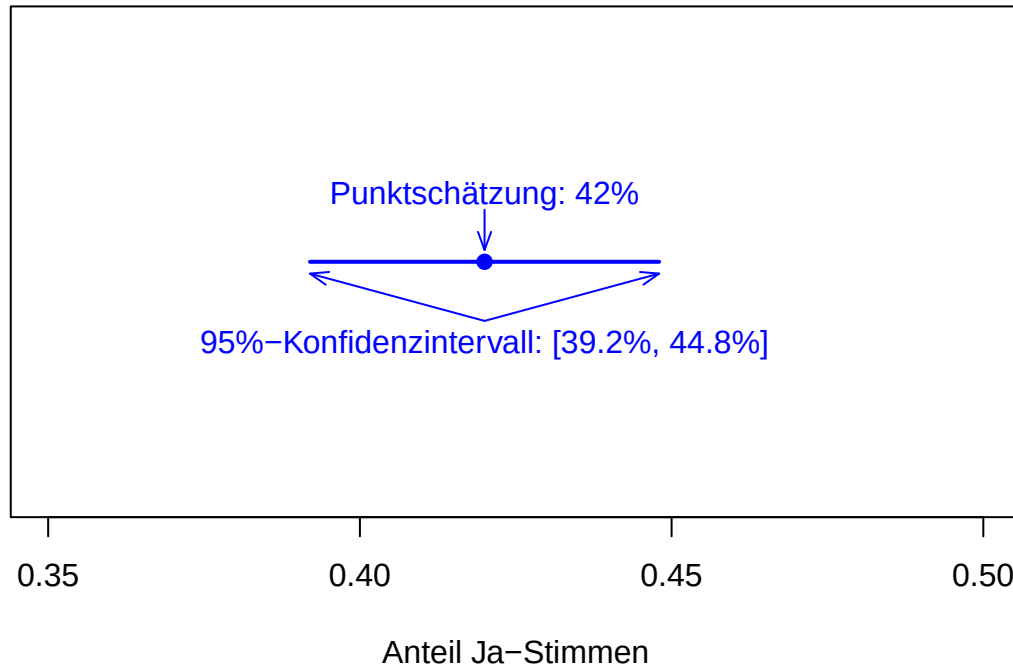
Ein Bereich plausibler Werte (oft als Konfidenzintervall bezeichnet) gibt an, in welchem Bereich der wahre Populationsparameter mit einer bestimmten Wahrscheinlichkeit liegt.

```
# Beispiel: SRF-Umfrage am 9. März 2021
#  $\hat{p} = 42\% \pm 2.8\%$  → plausible Werte: [0.392, 0.448]
p_hat <- 0.42
error <- 0.028
plausible_interval <- c(p_hat - error, p_hat + error)
cat("Bereich der plausiblen Werte (SRF-Umfrage):", round(plausible_interval[1], 3),
    "bis", round(plausible_interval[2], 3), "\n")
```

```
## Bereich der plausiblen Werte (SRF-Umfrage): 0.392 bis 0.448
```

```
# Visualisierung des Bereichs plausibler Werte
plot(c(0.35, 0.5), c(0, 0),
     type = "n",
     xlab = "Anteil Ja-Stimmen",
     ylab = "",
     main = "Bereich plausibler Werte für den Anteil Ja-Stimmen",
     yaxt = "n") # Keine y-Achse
points(p_hat, 0, pch = 19, col = "blue")
segments(plausible_interval[1], 0, plausible_interval[2], 0, col = "blue", lwd = 2)
text(p_hat, 0.28, "Punktschätzung: 42%", col = "blue")
text(mean(plausible_interval), -0.35, "95%-Konfidenzintervall: [39.2%, 44.8%]", col =
  ↪ "blue")
arrows(p_hat, 0.22, p_hat, 0.05, col = "blue", angle = 20, length = 0.1)
arrows(mean(plausible_interval), -0.25, plausible_interval[1], -0.05, col = "blue", angle
  ↪ = 20, length = 0.1)
arrows(mean(plausible_interval), -0.25, plausible_interval[2], -0.05, col = "blue", angle
  ↪ = 20, length = 0.1)
```

Bereich plausibler Werte für den Anteil Ja–Stimmen



Statistische Erklärung:

- Ein **Konfidenzintervall** ist ein Bereich, der mit einer bestimmten Wahrscheinlichkeit (z.B. 95%) den wahren Populationsparameter enthält.
- Die allgemeine Form eines Konfidenzintervalls ist: Punktschätzung $\pm$ Fehlerbereich.
- Der Fehlerbereich hängt vom gewünschten Konfidenzniveau, dem Standardfehler und der Verteilung der Stichprobenstatistik ab.
- Für den Mittelwert bei grossem n ist der Fehlerbereich ungefähr: $z * SE$, wobei z der z -Wert für das gewünschte Konfidenzniveau ist (z.B. 1.96 für 95%).
- Die Interpretation eines 95%-Konfidenzintervalls ist: Wenn wir viele Stichproben ziehen und jeweils ein 95%-Konfidenzintervall berechnen, werden etwa 95% dieser Intervalle den wahren Populationsparameter enthalten.

R-Erklärung:

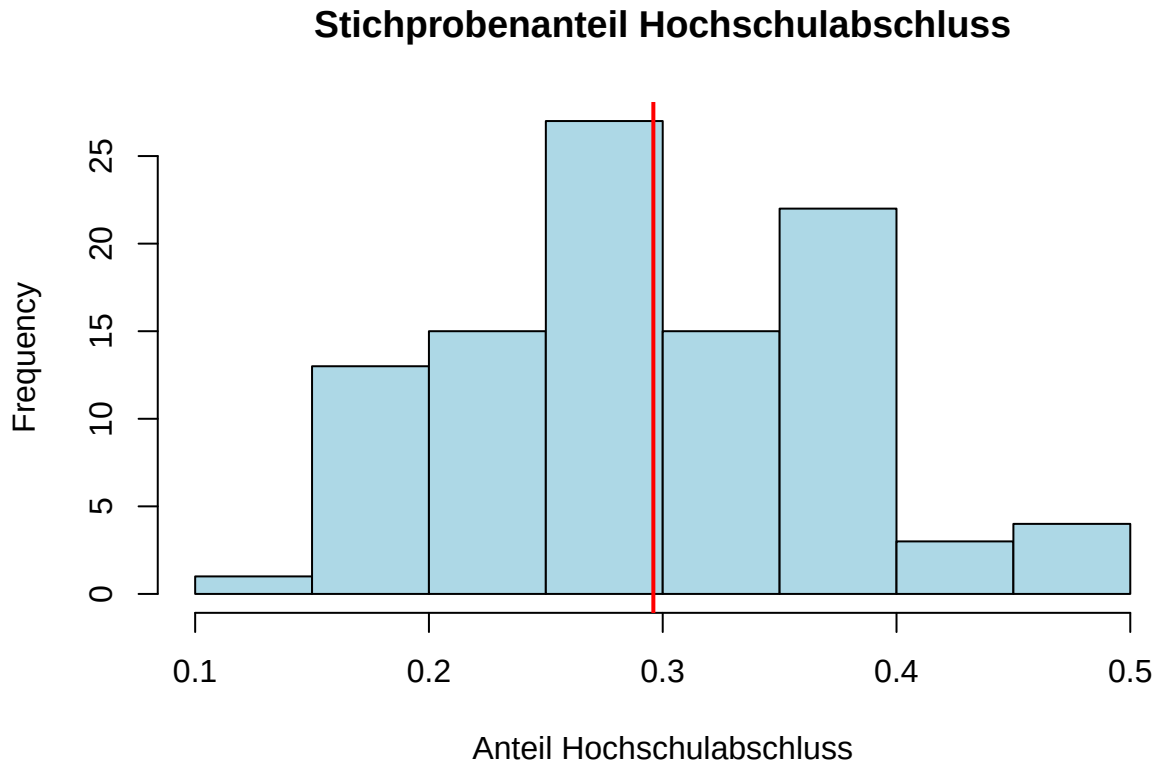
- Die Funktion `plot()` mit `type = "n"` erstellt einen leeren Plot.
- Die Funktion `points()` fügt Punkte zu einem bestehenden Plot hinzu.
- Die Funktion `segments()` fügt Liniensegmente zu einem bestehenden Plot hinzu.
- Der Parameter `yaxt = "n"` in `plot()` unterdrückt die Darstellung der y-Achse.

8 Konfidenzintervalle – Simulation für Proportionen

Wir können die Eigenschaften von Konfidenzintervallen durch Simulation untersuchen. Hier simulieren wir den Anteil von Personen mit Hochschulabschluss in der Schweiz.

```
# Beispiel: Anteil Hochschulabschluss bei Schweizerinnen (p = 0.296, d.h. 29.6%)
set.seed(101)
# Ziehe 100 Stichproben und berechne jeweils den Anteil
stich_m <- do(100) * mean(sample(0:1, prob = c(0.704, 0.296), size = 30, replace = TRUE))
```

```
# Visualisierung der Verteilung der Stichprobenanteile
hist(stich_m$mean,
     main = "Stichprobenanteil Hochschulabschluss",
     xlab = "Anteil Hochschulabschluss",
     col = "lightblue")
abline(v = 0.296, col = "red", lwd = 2)
```



```
# Berechnung des Standardfehlers
se_hat <- sd(stich_m$mean)
cat("Simulierter Standardfehler (SE) für den Anteil:", round(se_hat, 4), "\n")
```

```
## Simulierter Standardfehler (SE) für den Anteil: 0.0796
```

```
# Theoretischer Standardfehler für Proportionen: SE = sqrt(p*(1-p)/n)
p <- 0.296
n <- 30
se_theo <- sqrt(p * (1 - p) / n)
cat("Theoretischer Standardfehler (SE) für den Anteil:", round(se_theo, 4), "\n")
```

```
## Theoretischer Standardfehler (SE) für den Anteil: 0.0833
```

Statistische Erklärung:

- Für Proportionen (Anteile) gilt: Der theoretische Standardfehler ist $SE = \sqrt{p(1-p)/n}$, wobei p der wahre Anteil und n die Stichprobengröße ist.
- Die Stichprobenverteilung einer Proportion ist bei ausreichend grossem n annähernd normalverteilt (Binomialverteilung nähert sich der Normalverteilung).
- Als Faustregel gilt: Die Normalverteilungsapproximation ist angemessen, wenn $np \geq 5$ und $n(1-p) \geq 5$.

- In der Praxis kennen wir p oft nicht und müssen den Standardfehler mit $\sqrt{(\hat{p}^*(1-\hat{p}))/n}$ schätzen, wobei $\hat{p}$ der Stichprobenanteil ist.

R-Erklärung:

- Die Funktion `sample()` mit `replace = TRUE` zieht eine Stichprobe mit Zurücklegen.
- Der Parameter `prob` in `sample()` gibt die Wahrscheinlichkeiten für die verschiedenen Werte an.

9 Darstellung von Konfidenzintervallen aus der Simulation

Wir können für jede simulierte Stichprobe ein Konfidenzintervall berechnen und visualisieren, wie viele dieser Intervalle den wahren Populationsparameter enthalten.

```
# Berechne für jede der 100 Stichproben ein 95%-Konfidenzintervall ( $\bar{x} \pm 2 \cdot SE$ )
untere <- stich_m$mean - 2 * se_hat
obere <- stich_m$mean + 2 * se_hat

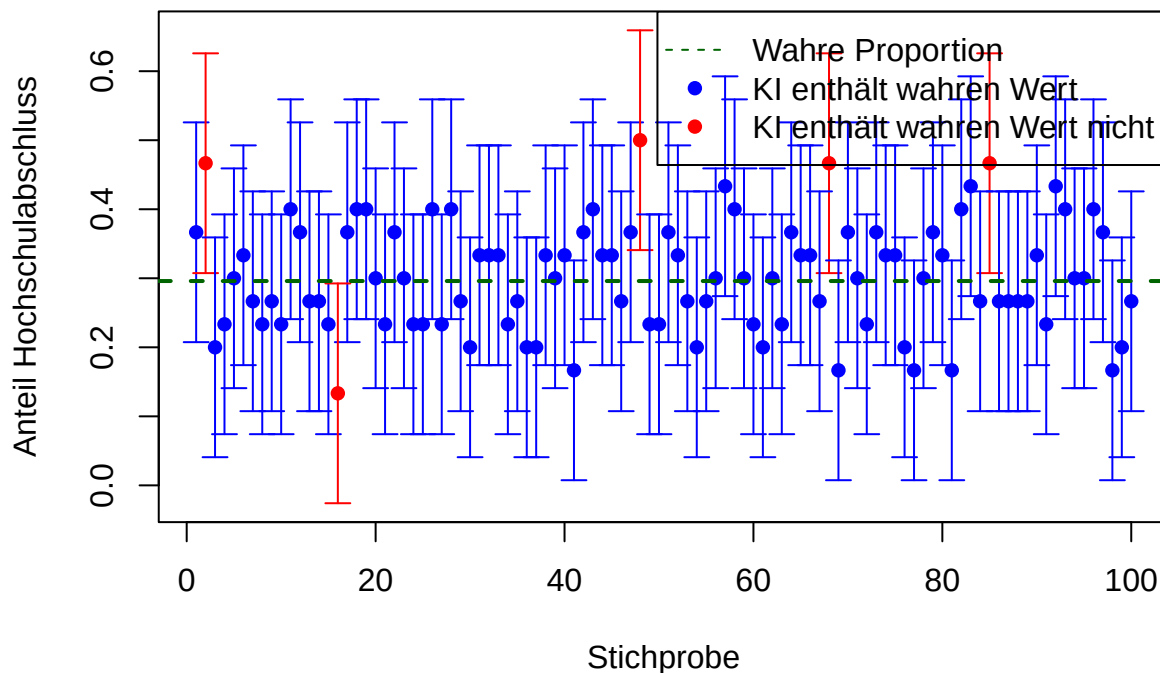
# Wahre Proportion
p_true <- 0.296

# Zähle, wie viele Intervalle den wahren Wert enthalten
enthält_p <- (untere <= p_true) & (p_true <= obere)
anteil_enthält_p <- mean(enthält_p)
cat("Anteil der Konfidenzintervalle, die den wahren Wert enthalten:",
    round(anteil_enthält_p * 100, 1), "%\n")
```

```
## Anteil der Konfidenzintervalle, die den wahren Wert enthalten: 95 %
```

```
# Visualisierung der Konfidenzintervalle
plotCI(x = 1:100, y = stich_m$mean, li = untere, ui = obere,
       main = "Konfidenzintervalle aus der Simulation",
       xlab = "Stichprobe", ylab = "Anteil Hochschulabschluss",
       col = ifelse(enthält_p, "blue", "red"),
       pch = 16)
abline(h = p_true, col = "darkgreen", lwd = 2, lty = 2)
legend("topright",
       legend = c("Wahre Proportion", "KI enthält wahren Wert", "KI enthält wahren Wert  
↪ nicht"),
       col = c("darkgreen", "blue", "red"),
       lty = c(2, NA, NA),
       pch = c(NA, 16, 16))
```

Konfidenzintervalle aus der Simulation



Statistische Erklärung:

- Ein **95%-Konfidenzintervall** bedeutet, dass bei wiederholter Stichprobenziehung etwa 95% der berechneten Intervalle den wahren Populationsparameter enthalten.
- Die Breite eines Konfidenzintervalls hängt ab von:
 - Dem gewählten Konfidenzniveau (höheres Niveau → breiteres Intervall)
 - Dem Standardfehler (größerer SE → breiteres Intervall)
 - Der Stichprobengröße (größerer n → schmaleres Intervall)
- Konfidenzintervalle geben einen Eindruck von der Präzision einer Schätzung: Je schmäler das Intervall, desto präziser die Schätzung.
- Die korrekte Interpretation eines einzelnen 95%-Konfidenzintervalls ist: “Wir sind zu 95% sicher, dass das berechnete Intervall den wahren Populationsparameter enthält.”

R-Erklärung:

- Die Funktion `plotCI()` aus dem Paket `plotrix` erstellt einen Plot mit Konfidenzintervallen.
- Die Parameter `li` und `ui` in `plotCI()` geben die untere bzw. obere Grenze der Konfidenzintervalle an.
- Die Funktion `ifelse()` gibt je nach Bedingung unterschiedliche Werte zurück, hier für die Farbe der Punkte.

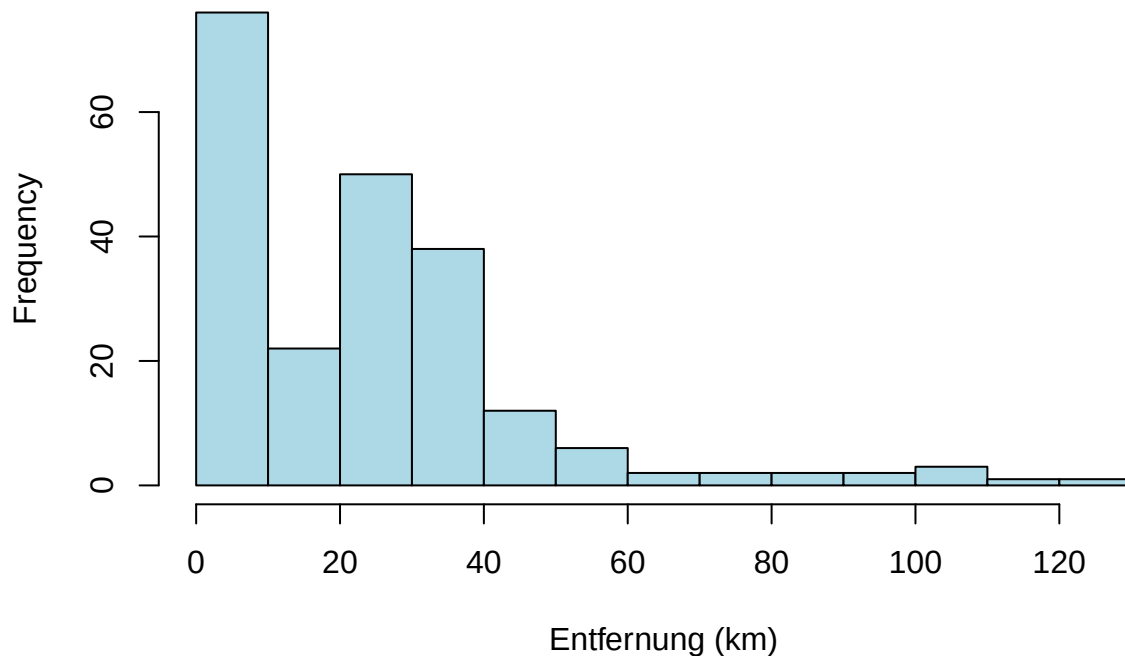
10 Bootstrap-Stichproben und Standardfehler

Die Bootstrap-Methode ist eine Resampling-Technik, bei der wiederholt Stichproben mit Zurücklegen aus der ursprünglichen Stichprobe gezogen werden. Sie ermöglicht die Schätzung von Standardfehlern und Konfidenzintervallen ohne Annahmen über die zugrundeliegende Verteilung.

```
# Beispiel: BFH-Datensatz, Variable distance (Arbeitsweg)
hist(BFH$distance,
     main = "Histogramm der Arbeitswege",
```

```
xlab = "Entfernung (km)",
col = "lightblue")
```

Histogramm der Arbeitswege



```
# Berechnung des Mittelwerts und der Standardabweichung
mean_distance <- mean(BFH$distance)
sd_distance <- sd(BFH$distance)
cat("Mittelwert der Arbeitswege:", round(mean_distance, 2), "km\n")
```

```
## Mittelwert der Arbeitswege: 25.2 km
```

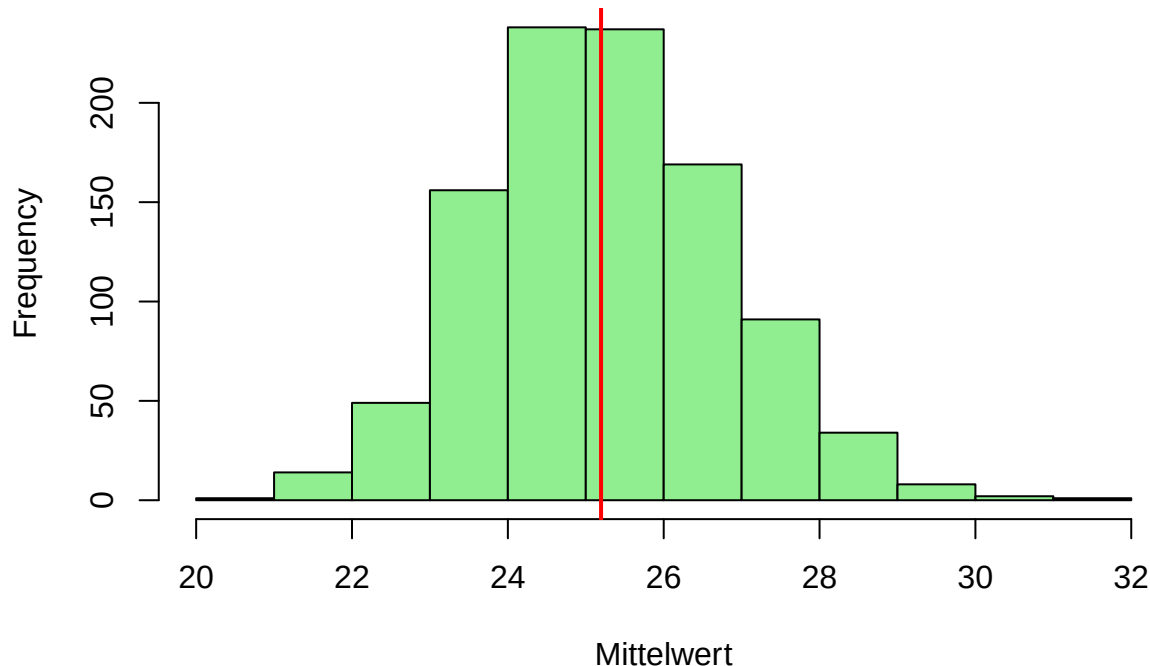
```
cat("Standardabweichung der Arbeitswege:", round(sd_distance, 2), "km\n")
```

```
## Standardabweichung der Arbeitswege: 22.65 km
```

```
# Ziehe 1000 Bootstrap-Stichproben und berechne jeweils den Mittelwert
set.seed(202)
boot1000.dist <- do(1000) * resample(BFH$distance)
boot1000.m <- apply(boot1000.dist, 1, mean)
```

```
# Visualisierung der Bootstrap-Verteilung
hist(boot1000.m,
     main = "Bootstrap-Verteilung der Mittelwerte (distance)",
     xlab = "Mittelwert",
     col = "lightgreen")
abline(v = mean_distance, col = "red", lwd = 2)
```

Bootstrap-Verteilung der Mittelwerte (distance)



```
# Berechnung des Bootstrap-Standardfehlers
dist.se.hat <- sd(boot1000.m)
cat("Bootstrap-Standardfehler (distance):", round(dist.se.hat, 2), "km\n")
```

```
## Bootstrap-Standardfehler (distance): 1.56 km
```

```
# Theoretischer Standardfehler zum Vergleich
n_bfh <- length(BFH$distance)
se_theo_dist <- sd_distance / sqrt(n_bfh)
cat("Theoretischer Standardfehler (distance):", round(se_theo_dist, 2), "km\n")
```

```
## Theoretischer Standardfehler (distance): 1.54 km
```

Statistische Erklärung:

- Die **Bootstrap-Methode** wurde 1979 von Bradley Efron entwickelt und ist eine computerintensive Methode zur Schätzung der Stichprobenverteilung einer Statistik.
- Grundidee: Die ursprüngliche Stichprobe wird als "Ersatzpopulation" betrachtet, aus der wiederholt Stichproben mit Zurücklegen gezogen werden.
- Für jede Bootstrap-Stichprobe wird die interessierende Statistik (z.B. Mittelwert) berechnet.
- Die Verteilung dieser Bootstrap-Statistiken approximiert die Stichprobenverteilung der Statistik.
- Der Standardfehler wird als Standardabweichung der Bootstrap-Statistiken geschätzt.
- Vorteile der Bootstrap-Methode:
 - Keine Annahmen über die zugrundeliegende Verteilung erforderlich
 - Anwendbar auf komplexe Statistiken, für die keine einfachen Formeln existieren
 - Funktioniert auch bei kleinen Stichproben

R-Erklärung:

- Die Funktion `resample()` aus dem Paket `mosaic` zieht eine Stichprobe mit Zurücklegen aus einem

Vektor oder Dataframe.

- Die Funktion `apply()` wendet eine Funktion auf die Zeilen ($\text{MARGIN} = 1$) oder Spalten ($\text{MARGIN} = 2$) einer Matrix an.

11 Bootstrap: Beispiel – Nussmischung

Ein weiteres Beispiel für die Bootstrap-Methode: Wir schätzen den Anteil von Cashew-Nüssen in einer Nussmischung.

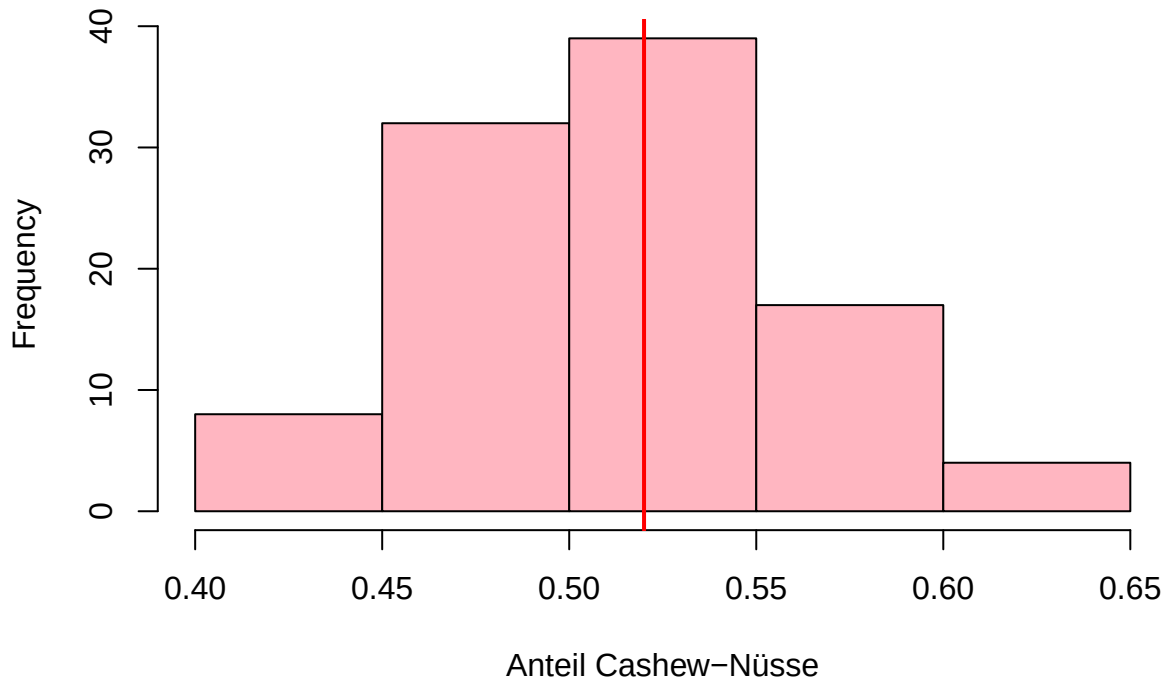
```
# Beispiel: Eine Packung Nussmischung enthält 100 Nüsse, davon 52 Cashew-Nüsse.
set.seed(303)
phat <- 0.52
nuts <- sample(c(rep(1, phat * 100), rep(0, (1 - phat) * 100)))
table(nuts) # Überprüfen der Häufigkeiten
```

```
## nuts
##  0  1
## 48 52
```

```
# Ziehe 100 Bootstrap-Stichproben und berechne jeweils den Anteil
nuts.re100 <- do(100) * resample(nuts)
nuts.m100 <- apply(nuts.re100, 1, mean)

# Visualisierung der Bootstrap-Verteilung
hist(nuts.m100,
     main = "Bootstrap-Verteilung des Anteils (Nussmischung)",
     xlab = "Anteil Cashew-Nüsse",
     col = "lightpink")
abline(v = phat, col = "red", lwd = 2)
```

Bootstrap-Verteilung des Anteils (Nussmischung)



```
# Berechnung des Bootstrap-Standardfehlers
nuts.se.hat <- sd(nuts.m100)
cat("Bootstrap-Standardfehler (Nussanteil):", round(nuts.se.hat, 3), "\n")
```

```
## Bootstrap-Standardfehler (Nussanteil): 0.048
```

```
# Theoretischer Standardfehler für Proportionen zum Vergleich
se_theo_nuts <- sqrt(phat * (1 - phat) / 100)
cat("Theoretischer Standardfehler (Nussanteil):", round(se_theo_nuts, 3), "\n")
```

```
## Theoretischer Standardfehler (Nussanteil): 0.05
```

Statistische Erklärung:

- Bei der Schätzung von Proportionen (Anteilen) ist die Bootstrap-Methode besonders nützlich, wenn die Stichprobengröße klein ist oder die Annahmen für die Normalverteilungsapproximation nicht erfüllt sind.
- Der Bootstrap-Standardfehler sollte nahe am theoretischen Standardfehler liegen, wenn die Stichprobengröße ausreichend gross ist.
- Die Bootstrap-Verteilung einer Proportion ist oft nicht symmetrisch, besonders wenn der Anteil nahe bei 0 oder 1 liegt.
- In solchen Fällen können Bootstrap-Konfidenzintervalle basierend auf Perzentilen genauer sein als Intervalle basierend auf dem Standardfehler.

R-Erklärung:

- Die Funktion `rep()` wiederholt einen Wert oder Vektor.
- Die Funktion `table()` erstellt eine Häufigkeitstabelle.

12 Konfidenzintervalle mit der Bootstrap-Methode (SE-Methode)

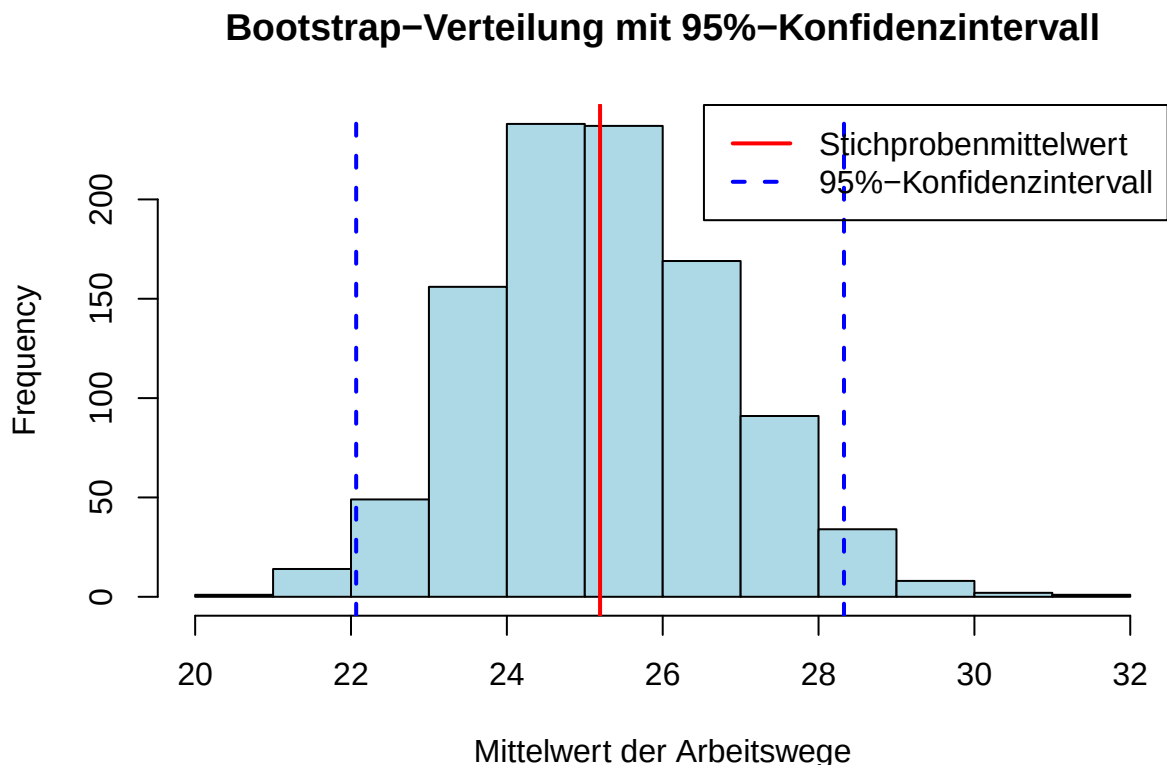
Mit dem Bootstrap-Standardfehler können wir Konfidenzintervalle für den Mittelwert der Arbeitswege berechnen.

```
# Berechnung des Mittelwerts der Arbeitswege
xbar_distance <- mean(BFH$distance)

# Berechnung des 95%-Konfidenzintervalls mit der SE-Methode
# 95%-KI:  $\bar{x} \pm 1.96 \cdot SE$  (oft gerundet auf  $\bar{x} \pm 2 \cdot SE$ )
dist.konf <- xbar_distance + c(-1, 1) * 2 * dist.se.hat
cat("95%-Konfidenzintervall (mit SE-Methode) für distance:", round(dist.konf[1], 2),
    "bis", round(dist.konf[2], 2), "km\n")
```

```
## 95%-Konfidenzintervall (mit SE-Methode) für distance: 22.07 bis 28.33 km
```

```
# Visualisierung des Konfidenzintervalls
hist(boot1000.m,
     main = "Bootstrap-Verteilung mit 95%-Konfidenzintervall",
     xlab = "Mittelwert der Arbeitswege",
     col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = dist.konf, col = "blue", lwd = 2, lty = 2)
legend("topright",
     legend = c("Stichprobenmittelwert", "95%-Konfidenzintervall"),
     col = c("red", "blue"),
     lty = c(1, 2),
     lwd = 2)
```



Statistische Erklärung:

- Die **SE-Methode** für Bootstrap-Konfidenzintervalle basiert auf der Annahme, dass die Bootstrap-Verteilung annähernd normalverteilt ist.
- Das 95%-Konfidenzintervall wird berechnet als: $\bar{x} \pm 1.96 \cdot SE$, wobei SE der Bootstrap-Standardfehler ist.
- Diese Methode ist einfach anzuwenden und funktioniert gut, wenn die Bootstrap-Verteilung symmetrisch ist.
- Bei schiefen Verteilungen oder kleinen Stichproben kann die SE-Methode jedoch ungenau sein.
- In solchen Fällen sind Perzentil-basierte Methoden oft besser geeignet.

R-Erklärung:

- Der Ausdruck `c(-1, 1)` erzeugt einen Vektor mit den Werten -1 und 1.
- Die Multiplikation mit `2 * dist.se.hat` und Addition zu `xbar_distance` erzeugt die untere und obere Grenze des Konfidenzintervalls.

13 Alternative Konfidenzintervalle – Bootstrap-Perzentil-Methode

Eine alternative Methode zur Berechnung von Bootstrap-Konfidenzintervallen ist die Perzentil-Methode, bei der direkt die Perzentile der Bootstrap-Verteilung verwendet werden.

```
# Berechnung des 95%-Konfidenzintervalls mit der Perzentil-Methode
dist.q025 <- quantile(boot1000.m, probs = 0.025, type = 1)
dist.q975 <- quantile(boot1000.m, probs = 0.975, type = 1)
cat("95%-Konfidenzintervall (Perzentil-Methode) für distance:", round(dist.q025, 2),
    "bis", round(dist.q975, 2), "km\n")
```

```
## 95%-Konfidenzintervall (Perzentil-Methode) für distance: 22.29 bis 28.43 km
```

```
# Vergleich der beiden Methoden
cat("SE-Methode:      ", round(dist.konf[1], 2), "bis", round(dist.konf[2], 2), "km\n")
```

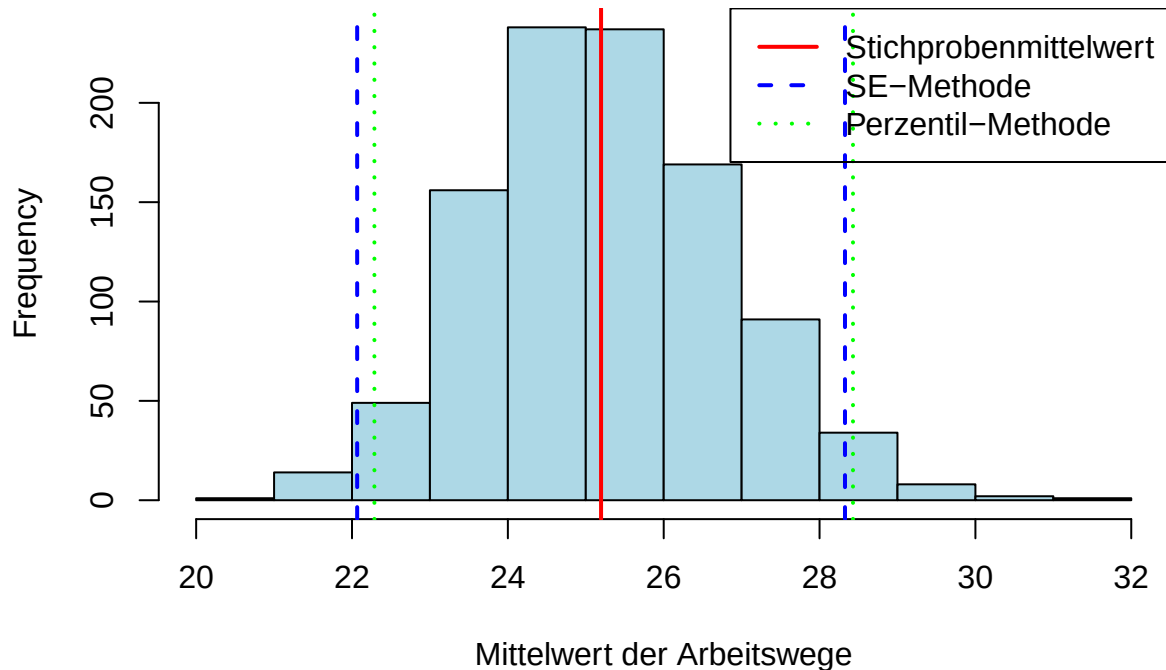
```
## SE-Methode:      22.07 bis 28.33 km
```

```
cat("Perzentil-Methode:", round(dist.q025, 2), "bis", round(dist.q975, 2), "km\n")
```

```
## Perzentil-Methode: 22.29 bis 28.43 km
```

```
# Visualisierung beider Konfidenzintervalle
hist(boot1000.m,
     main = "Vergleich der Konfidenzintervall-Methoden",
     xlab = "Mittelwert der Arbeitswege",
     col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = dist.konf, col = "blue", lwd = 2, lty = 2)
abline(v = c(dist.q025, dist.q975), col = "green", lwd = 2, lty = 3)
legend("topright",
     legend = c("Stichprobenmittelwert", "SE-Methode", "Perzentil-Methode"),
     col = c("red", "blue", "green"),
     lty = c(1, 2, 3),
     lwd = 2)
```

Vergleich der Konfidenzintervall-Methoden



Statistische Erklärung:

- Die **Perzentil-Methode** für Bootstrap-Konfidenzintervalle verwendet direkt die Perzentile der Bootstrap-Verteilung.
- Ein 95%-Konfidenzintervall wird durch das 2.5- und 97.5-Perzentil der Bootstrap-Verteilung definiert.
- Vorteile der Perzentil-Methode:
 - Berücksichtigt die Form der Bootstrap-Verteilung
 - Funktioniert auch bei schiefen Verteilungen
 - Einfach zu berechnen und zu interpretieren
- Nachteile:
 - Kann bei kleinen Stichproben ungenau sein
 - Berücksichtigt nicht die Verzerrung der Bootstrap-Verteilung
- Bei symmetrischen Verteilungen liefern die SE-Methode und die Perzentil-Methode ähnliche Ergebnisse.
- Bei schiefen Verteilungen können die Ergebnisse deutlich unterschiedlich sein.

R-Erklärung:

- Die Funktion `quantile()` berechnet Quantile (Perzentile) eines Vektors.
- Der Parameter `probs` gibt die gewünschten Perzentile als Dezimalzahlen an.
- Der Parameter `type` gibt die Methode zur Berechnung der Quantile an (hier: Typ 1).

14 Konfidenzintervalle mit Perzentilen für unterschiedliche Konfidenzniveaus

Wir können Bootstrap-Konfidenzintervalle für verschiedene Konfidenzniveaus berechnen, um die Auswirkung des Konfidenzniveaus auf die Breite des Intervalls zu untersuchen.

```
# 90%-Konfidenzintervall
dist.q05 <- quantile(boot1000.m, probs = 0.05, type = 1)
```

```
dist.q95 <- quantile(boot1000.m, probs = 0.95, type = 1)
cat("90%-Konfidenzintervall für distance:", round(dist.q05, 2), "bis", round(dist.q95,
  ↪ 2), "km\n")
```

```
## 90%-Konfidenzintervall für distance: 22.81 bis 27.92 km
```

```
# 95%-Konfidenzintervall (bereits berechnet)
cat("95%-Konfidenzintervall für distance:", round(dist.q025, 2), "bis", round(dist.q975,
  ↪ 2), "km\n")
```

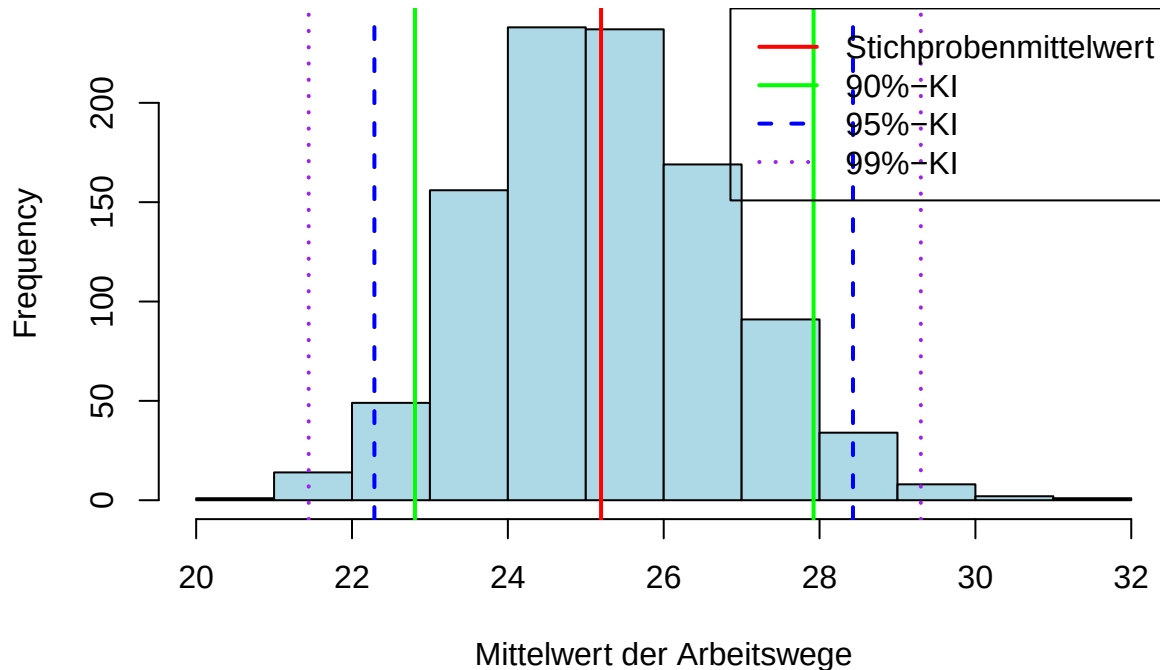
```
## 95%-Konfidenzintervall für distance: 22.29 bis 28.43 km
```

```
# 99%-Konfidenzintervall
dist.q005 <- quantile(boot1000.m, probs = 0.005, type = 1)
dist.q995 <- quantile(boot1000.m, probs = 0.995, type = 1)
cat("99%-Konfidenzintervall für distance:", round(dist.q005, 2), "bis", round(dist.q995,
  ↪ 2), "km\n")
```

```
## 99%-Konfidenzintervall für distance: 21.44 bis 29.3 km
```

```
# Visualisierung der verschiedenen Konfidenzintervalle
hist(boot1000.m,
  main = "Konfidenzintervalle für verschiedene Konfidenzniveaus",
  xlab = "Mittelwert der Arbeitswege",
  col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = c(dist.q05, dist.q95), col = "green", lwd = 2, lty = 1)
abline(v = c(dist.q025, dist.q975), col = "blue", lwd = 2, lty = 2)
abline(v = c(dist.q005, dist.q995), col = "purple", lwd = 2, lty = 3)
legend("topright",
  legend = c("Stichprobenmittelwert", "90%-KI", "95%-KI", "99%-KI"),
  col = c("red", "green", "blue", "purple"),
  lty = c(1, 1, 2, 3),
  lwd = 2)
```

Konfidenzintervalle für verschiedene Konfidenzniveaus



Statistische Erklärung:

- Das **Konfidenzniveau** gibt die Wahrscheinlichkeit an, mit der das berechnete Intervall den wahren Populationsparameter enthält.
- Höhere Konfidenzniveaus führen zu breiteren Intervallen:
 - 90%-KI: Verwendet das 5. und 95. Perzentil
 - 95%-KI: Verwendet das 2.5. und 97.5. Perzentil
 - 99%-KI: Verwendet das 0.5. und 99.5. Perzentil
- Die Wahl des Konfidenzniveaus hängt vom Anwendungskontext ab:
 - Höhere Niveaus (z.B. 99%) bieten mehr Sicherheit, aber weniger Präzision
 - Niedrigere Niveaus (z.B. 90%) bieten mehr Präzision, aber weniger Sicherheit
- In der Praxis wird häufig ein Konfidenzniveau von 95% verwendet, was einen guten Kompromiss zwischen Sicherheit und Präzision darstellt.

R-Erklärung:

- Die verschiedenen Linientypen (`lty`) helfen, die verschiedenen Konfidenzintervalle visuell zu unterscheiden.

15 Konfidenzintervalle für Differenzen

Wir können die Bootstrap-Methode auch verwenden, um Konfidenzintervalle für Differenzen zwischen Gruppen zu berechnen, z.B. für die Differenz der durchschnittlichen Trainingszeiten zwischen Männern und Frauen.

```
# Beispiel: Differenz der durchschnittlichen Exercise-Stunden zwischen Männern und Frauen
men <- ExerciseHours[ExerciseHours$Sex == "M", ]
women <- ExerciseHours[ExerciseHours$Sex == "F", ]

# Berechnung der Mittelwerte und der Differenz
```

```

mw.m <- mean(men$Exercise)
mw.f <- mean(women$Exercise)
diff.hat <- mw.m - mw.f
cat("Mittelwert Männer:", round(mw.m, 2), "Stunden\n")

## Mittelwert Männer: 12.4 Stunden

cat("Mittelwert Frauen:", round(mw.f, 2), "Stunden\n")

## Mittelwert Frauen: 9.4 Stunden

cat("Differenz (Männer - Frauen):", round(diff.hat, 2), "Stunden\n")

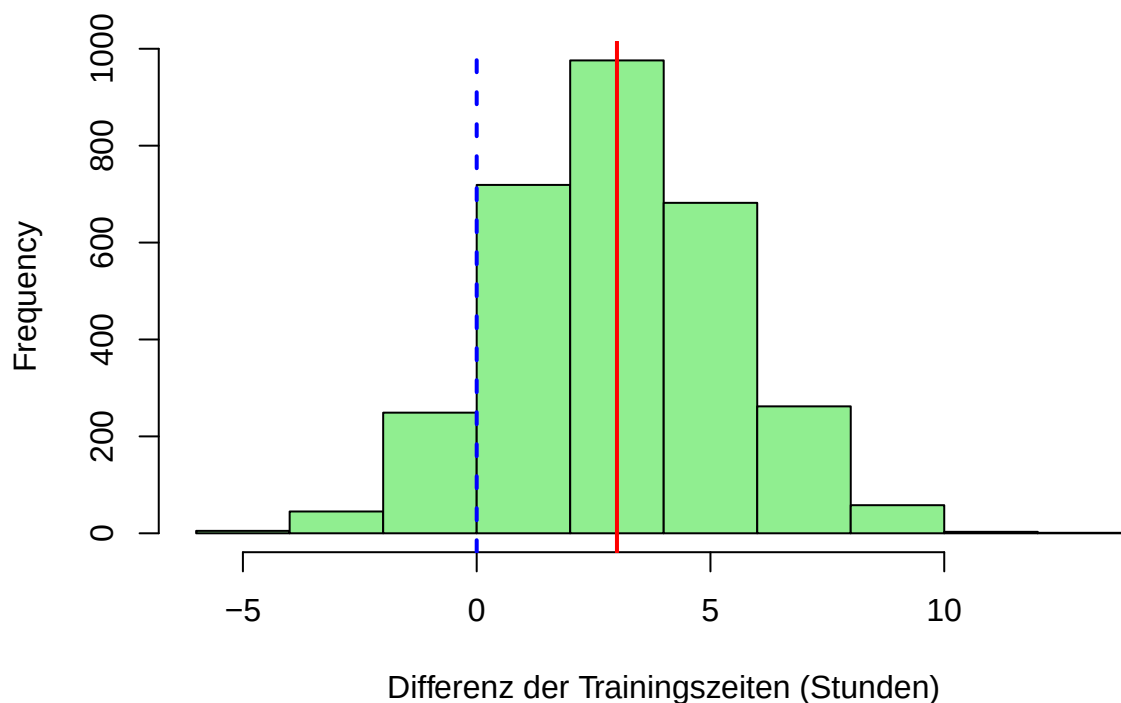
## Differenz (Männer - Frauen): 3 Stunden

# Bootstrap für Differenzen: Ziehe 3000 Bootstrap-Stichproben
set.seed(404)
boot3000.diff <- do(3000) * (mean(resample(men$Exercise)) -
  ↪ mean(resample(women$Exercise)))

# Visualisierung der Bootstrap-Verteilung der Differenz
hist(boot3000.diff$result,
     main = "Bootstrap-Verteilung der Differenz (Männer - Frauen)",
     xlab = "Differenz der Trainingszeiten (Stunden)",
     col = "lightgreen")
abline(v = diff.hat, col = "red", lwd = 2)
abline(v = 0, col = "blue", lwd = 2, lty = 2)

```

Bootstrap-Verteilung der Differenz (Männer – Frauen)



```
# Berechnung des 95%-Konfidenzintervalls mit der Perzentil-Methode
diff.q025 <- quantile(boot3000.diff$result, probs = 0.025, type = 1)
diff.q975 <- quantile(boot3000.diff$result, probs = 0.975, type = 1)
cat("95%-Konfidenzintervall für die Differenz:", round(diff.q025, 2), "bis",
    ↪ round(diff.q975, 2), "Stunden\n")

## 95%-Konfidenzintervall für die Differenz: -1.58 bis 7.75 Stunden

# Überprüfen, ob das Konfidenzintervall die 0 enthält
enthält_null <- (diff.q025 <= 0) & (0 <= diff.q975)
cat("Enthält das Konfidenzintervall die 0?", ifelse(enthält_null, "Ja", "Nein"), "\n")

## Enthält das Konfidenzintervall die 0? Ja

cat("Interpretation: Die Differenz ist", ifelse(enthält_null, "nicht", ""), "statistisch
    ↪ signifikant.\n")

## Interpretation: Die Differenz ist nicht statistisch signifikant.
```

Statistische Erklärung:

- Konfidenzintervalle für **Differenzen** sind besonders nützlich, um Unterschiede zwischen Gruppen zu beurteilen.
- Die Bootstrap-Methode für Differenzen umfasst folgende Schritte:
 1. Ziehe Bootstrap-Stichproben aus beiden Gruppen
 2. Berechne für jedes Paar von Bootstrap-Stichproben die Differenz der Mittelwerte
 3. Verwende die Verteilung dieser Differenzen, um ein Konfidenzintervall zu berechnen
- Ein wichtiger Aspekt bei der Interpretation: Enthält das Konfidenzintervall die 0?
 - Wenn ja: Der Unterschied ist nicht statistisch signifikant (auf dem entsprechenden Niveau)
 - Wenn nein: Der Unterschied ist statistisch signifikant
- Diese Methode ist äquivalent zu einem Hypothesentest für den Unterschied zwischen den Gruppen.

R-Erklärung:

- Der Ausdruck `ExerciseHours[ExerciseHours$Sex == "M", ]` filtert die Zeilen des Dataframes, bei denen die Spalte `Sex` den Wert "M" hat.
- Die Funktion `ifelse()` gibt je nach Bedingung unterschiedliche Werte zurück, hier für die Interpretation des Konfidenzintervalls.

16 Konfidenzintervalle für Korrelationen

Schliesslich können wir die Bootstrap-Methode auch verwenden, um Konfidenzintervalle für Korrelationen zu berechnen, z.B. für die Korrelation zwischen Preis und Kilometerstand von Mustang-Autos.

```
# Beispiel: Bestimme die Korrelation zwischen Price und Miles im Mustangs-Datensatz
cor_mustangs <- cor(Mustangs$Price, Mustangs$Miles)
cat("Korrelation zwischen Price und Miles:", round(cor_mustangs, 3), "\n")
```

```
## Korrelation zwischen Price und Miles: -0.825
```

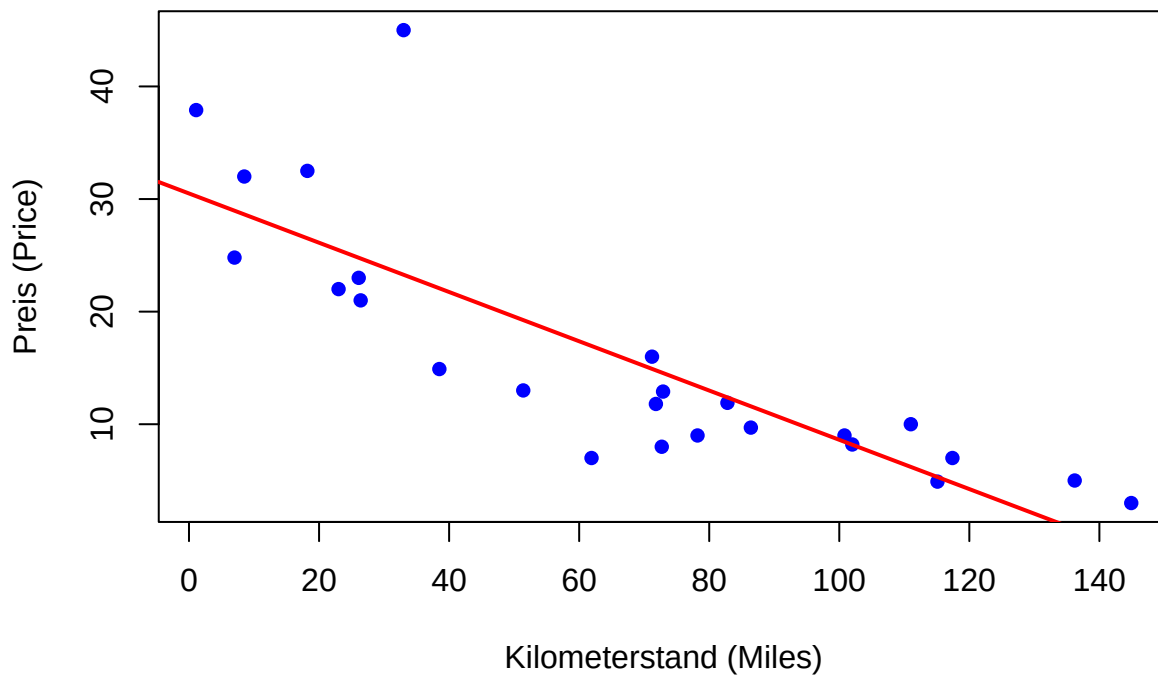
```
# Streudiagramm
plot(Mustangs$Miles, Mustangs$Price,
     main = "Streudiagramm: Preis vs. Kilometerstand",
```

```

xlab = "Kilometerstand (Miles)",
ylab = "Preis (Price)",
pch = 16,
col = "blue")
abline(lm(Price ~ Miles, data = Mustangs), col = "red", lwd = 2)

```

Streudiagramm: Preis vs. Kilometerstand



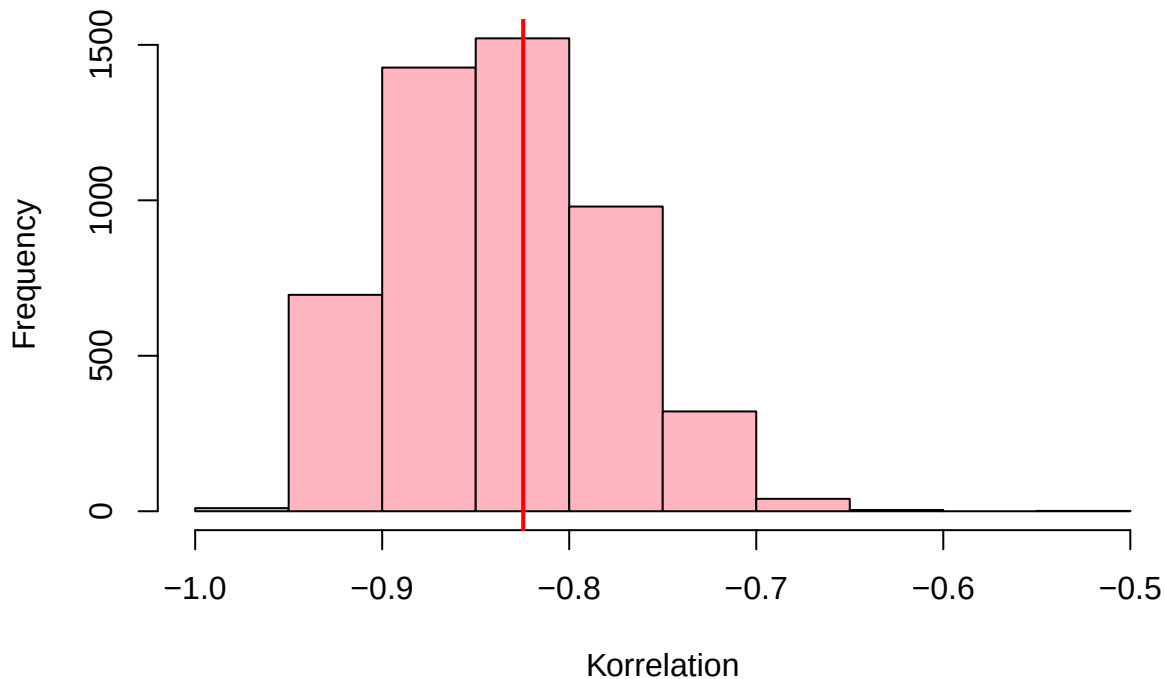
```

# Bootstrap für Korrelationen: Ziehe 5000 Bootstrap-Stichproben
set.seed(505)
mustangs.cor.boot <- do(5000) * cor(Price ~ Miles, data = resample(Mustangs))

# Visualisierung der Bootstrap-Verteilung der Korrelation
hist(mustangs.cor.boot$cor,
     main = "Bootstrap-Verteilung der Korrelation",
     xlab = "Korrelation",
     col = "lightpink")
abline(v = cor_mustangs, col = "red", lwd = 2)

```

Bootstrap-Verteilung der Korrelation



```
# Berechnung des 98%-Konfidenzintervalls mit der Perzentil-Methode
cor.q01 <- quantile(mustangs.cor.boot$cor, probs = 0.01, type = 1)
cor.q99 <- quantile(mustangs.cor.boot$cor, probs = 0.99, type = 1)
cat("98%-Konfidenzintervall für die Korrelation:", round(cor.q01, 3), "bis",
    ↪ round(cor.q99, 3), "\n")
```

```
## 98%-Konfidenzintervall für die Korrelation: -0.937 bis -0.703
```

```
# Alternative mit der Funktion qdata aus dem mosaic-Paket
quantiles <- qdata(~ cor, c(0.01, 0.99), data = mustangs.cor.boot)
cat("98%-Konfidenzintervall (mit qdata):\n")
```

```
## 98%-Konfidenzintervall (mit qdata):
```

```
print(quantiles)
```

```
##          1%          99%
## -0.9369749 -0.7025379
```

```
# Überprüfen, ob das Konfidenzintervall die 0 enthält
enthält_null <- (cor.q01 <= 0) & (0 <= cor.q99)
cat("Enthält das Konfidenzintervall die 0?", ifelse(enthält_null, "Ja", "Nein"), "\n")
```

```
## Enthält das Konfidenzintervall die 0? Nein
```

```
cat("Interpretation: Die Korrelation ist", ifelse(enthält_null, "nicht", ""),
    ↪ "statistisch signifikant.\n")
```

Interpretation: Die Korrelation ist statistisch signifikant.

Statistische Erklärung:

- Die **Korrelation** ist ein Mass für den linearen Zusammenhang zwischen zwei Variablen und liegt zwischen -1 und +1.
- Die Bootstrap-Methode für Korrelationen umfasst folgende Schritte:
 1. Ziehe Bootstrap-Stichproben aus dem Datensatz (Paare von Werten bleiben erhalten)
 2. Berechne für jede Bootstrap-Stichprobe die Korrelation
 3. Verwende die Verteilung dieser Korrelationen, um ein Konfidenzintervall zu berechnen
- Die Interpretation des Konfidenzintervalls ist ähnlich wie bei Differenzen:
 - Enthält das Intervall die 0? Wenn nein, ist die Korrelation statistisch signifikant
 - Die Breite des Intervalls gibt Aufschluss über die Präzision der Schätzung
- Die Bootstrap-Methode ist besonders nützlich für Korrelationen, da die Stichprobenverteilung der Korrelation oft nicht normalverteilt ist, besonders bei kleinen Stichproben oder extremen Korrelationen.

R-Erklärung:

- Die Formel `Price ~ Miles in cor()` gibt an, dass die Korrelation zwischen den Variablen `Price` und `Miles` berechnet werden soll.
- Die Funktion `qdata()` aus dem Paket `mosaic` berechnet Quantile für eine Variable in einem Dataframe.
- Die Funktion `lm()` passt ein lineares Modell an die Daten an, hier für die Regressionslinie im Streudiagramm.

17 Zusammenfassung

In dieser Vorlesung haben wir grundlegende Konzepte der schliessenden Statistik kennengelernt:

1. **Stichprobenvariation:** Verschiedene Stichproben aus derselben Population führen zu unterschiedlichen Schätzungen.
2. **Standardfehler:** Ein Mass für die Präzision einer Stichprobenstatistik, das mit zunehmender Stichprobengrösse abnimmt.
3. **Bootstrap-Methode:** Eine Resampling-Technik zur Schätzung von Standardfehlern und Konfidenzintervallen ohne Annahmen über die zugrundeliegende Verteilung.
4. **Konfidenzintervalle:** Bereiche, die mit einer bestimmten Wahrscheinlichkeit den wahren Populationsparameter enthalten.

Diese Konzepte sind grundlegend für die statistische Inferenz und ermöglichen es uns, von Stichproben auf die Population zu schliessen und die Unsicherheit unserer Schätzungen zu quantifizieren.

Die Bootstrap-Methode ist besonders nützlich, da sie:

- Keine Annahmen über die zugrundeliegende Verteilung erfordert
- Auf eine Vielzahl von Statistiken anwendbar ist
- Auch bei kleinen Stichproben funktioniert
- Einfach zu implementieren ist